

Parte 5: Laboratorio 5: Funcionalidades Avanzadas de P4

Validación académica – ECN en el plano de egreso y segmentación VLAN con marcado DSCP

Convención de colores

A lo largo de este documento se usa la misma convención visual de las validaciones previas para distinguir el origen de cada fragmento:

- Verde: comandos ejecutados en el shell Bash de la máquina virtual.
- Azul: fragmentos de scripts Python.
- Rojo: comandos ejecutados desde la CLI de Mininet o en terminales de hosts virtuales.
- Naranja: fragmentos de código P4 directamente relevantes para el comportamiento validado.

Introducción

Este documento constituye la validación académica del quinto laboratorio de la secuencia de prácticas de redes programables del curso Internet del Futuro. El laboratorio introduce dos mecanismos de red avanzados implementados directamente en el plano de datos mediante P4: la Notificación Explícita de Congestión (ECN) y la segmentación de red mediante VLAN 802.1Q con marcado DSCP por VLAN [1, 2].

Ambos ejercicios comparten el mismo flujo de trabajo de compilación e implementación establecido en el laboratorio anterior: compilación con p4c-bm2-ss, levantamiento de la topología con topo.py y carga de reglas de reenvío mediante simple_switch_CLI. Esta coherencia metodológica permite al estudiante

concentrarse en los conceptos de red que cada ejercicio introduce, sin distracciones derivadas de cambios en la infraestructura de desarrollo. En ambos casos, los ejercicios fueron adaptados de repositorios externos al entorno del curso, migrando desde infraestructuras basadas en P4Runtime/gRPC o Stratum hacia el uso de simple_switch y la CLI Thrift de BMv2, que requiere únicamente las dependencias mínimas de la máquina virtual validada en el laboratorio anterior.

El ejercicio guiado (ECN) presenta un programa P4 completamente funcional que detecta congestión de cola en el pipeline de egreso y la señala modificando los bits ECN del encabezado IPv4 sin descartar paquetes. La actividad del estudiante (VLAN) requiere completar un esqueleto de programa P4 que implementa etiquetado y desetiquetado 802.1Q, reenvío aislado por VLAN y marcado DSCP por VLAN.

Una diferencia conceptual importante entre ambos ejercicios es el modelo de reenvío y el punto del pipeline donde ocurre la modificación del paquete: el ejercicio ECN utiliza reenvío de capa 3 con coincidencia LPM sobre la dirección IPv4 destino y reescritura de MACs en cada salto, y el comportamiento central ocurre en el pipeline de egreso porque solo allí está disponible el metadato de profundidad de cola. El ejercicio VLAN utiliza reenvío de capa 2 y el comportamiento central ocurre en ingreso, donde la presencia o ausencia de la etiqueta VLAN y el puerto de llegada determinan qué transformación estructural debe aplicarse al paquete [1, 10].

Ejercicio 1: ECN – Notificación Explícita de Congestión (Guiado)

Contexto teórico

La Notificación Explícita de Congestión (ECN) es un mecanismo definido en la RFC 3168 que permite a los dispositivos de red señalar congestión a los extremos de una conexión sin necesidad de descartar paquetes [4]. A diferencia del descarte activo de paquetes (como RED), ECN marca el campo

correspondiente en el encabezado IP para que el receptor informe al emisor, quien reduce su tasa de envío de forma proactiva.

El campo ECN ocupa los dos bits menos significativos del byte TOS del encabezado IPv4, tal como lo define la RFC 3168 y la RFC 2474 [4, 5]. Los cuatro valores posibles son: 00 (no ECN-capable), 01 y 10 (ECN-Capable Transport, ECT) y 11 (Congestion Experienced, CE). Cuando un switch detecta que la profundidad de su cola supera un umbral, puede marcar el campo ECN de 01 o 10 a 11 (CE) en los paquetes ECN-capable, en lugar de descartarlos.

Implementar ECN en P4 obliga al estudiante a entender tres aspectos del pipeline v1model que no aparecen en ejercicios anteriores: la división del campo TOS en subcampos independientes (diffserv y ecn), el uso del metadato `standard_metadata.enq_qdepth`, definido por la arquitectura v1model e implementado en BMv2, cuyo valor solo está bien definido tras el encolamiento del paquete y por tanto solo es útil en el pipeline de egreso, y la necesidad de recalcular el checksum IPv4 cada vez que se modifica cualquier campo del encabezado IP [2].

Descripción de la topología y adaptaciones del ejercicio

El ejercicio está basado en el ejercicio ecn del repositorio oficial `p4lang/tutorials` [3]. La adaptación más significativa es la reducción de la topología de tres switches y cinco hosts a dos switches y cuatro hosts: la ruta alternativa del ejercicio original (un tercer switch no congestionado) es pedagógicamente marginal para el concepto de ECN, ya que un solo enlace cuello de botella compartido por múltiples flujos es suficiente para demostrar el marcado CE. En términos de infraestructura, el ejercicio original utiliza `simple_switch_grpc` como target BMv2 y archivos JSON en formato P4Runtime para las reglas de reenvío; esta adaptación emplea `simple_switch` y la CLI Thrift (`simple_switch_CLI`), eliminando completamente la dependencia de gRPC, protobuf y la librería cliente P4Runtime. Los scripts `send.py` y `receive.py`, basados en Scapy, se conservan del ejercicio original sin modificaciones. Cada instancia de `simple_switch` recibe un argumento `-device-id` único (0 para s1, 1 para s2) para evitar colisiones en los sockets IPC que BMv2 crea en `/tmp/`; sin este argumento, el segundo

switch sobrescribiría el socket del primero y la CLI no podría conectarse al switch correcto.

La topología consiste en dos switches P4 y cuatro hosts distribuidos en dos subredes. El enlace entre los switches actúa como cuello de botella limitado a 0,5 Mbps, lo que permite inducir congestión de forma controlada mediante un generador de tráfico de fondo.

La asignación de hosts y direccionamiento es la siguiente:

- s1 (puerto Thrift 9090): h1 en 10.0.1.1/24 en el puerto 1, h11 en 10.0.1.11/24 en el puerto 2, s2 en el puerto 3.
- s2 (puerto Thrift 9091): h2 en 10.0.2.2/24 en el puerto 1, h22 en 10.0.2.22/24 en el puerto 2, s1 en el puerto 3.

Dado que el ejercicio utiliza reenvío de capa 3, cada host está configurado con una ruta por defecto hacia una IP de gateway virtual y una entrada ARP estática que asocia dicha IP con la MAC del switch correspondiente. Esta configuración es aplicada automáticamente por la función `configure_hosts()` en `topo.py` y no requiere intervención manual. Los pares de hosts en la misma subred (h1–h11 en s1, h2–h22 en s2) no pueden comunicarse entre sí porque la tabla LPM no gestiona tráfico dentro de la misma subred; este comportamiento es correcto y no indica un error de configuración.

Paso 1 – Compilación del programa P4

El primer paso en cualquier ejercicio P4 con la infraestructura del curso es compilar el programa fuente al formato JSON que consume BMv2 (`simple_switch`). El compilador `p4c-bm2-ss` transforma el programa P4-16 en una representación intermedia que describe el pipeline completo: parser, controles de ingreso y egreso, cómputo de checksum y deparser [1, 2].

Desde el directorio del ejercicio ECN:

```
mkdir -p p4src/build
```

```
p4c-bm2-ss --p4v 16 -o p4src/build/bmv2.json \  
--p4runtime-files p4src/build/p4info.txt p4src/ecn.p4
```

Una compilación exitosa produce dos artefactos: p4src/build/bmv2.json, que contiene la descripción del pipeline en formato JSON consumible por simple_switch, y p4src/build/p4info.txt, que describe la interfaz de control del pipeline (tablas, acciones y sus parámetros). Un aviso [-Wwarn=deprecated] relacionado con el formato .txt del archivo p4info es esperado y no impide la ejecución. La confirmación definitiva de que la compilación fue exitosa se obtiene en el paso siguiente: topo.py carga explícitamente la ruta p4src/build/bmv2.json al arrancar cada switch, y si el archivo no existe o es inválido, BMv2 falla al iniciarse.

Paso 2 – Inicio de la topología Mininet

El script topo.py instancia la topología emulada mediante la API de Mininet. Para cada switch P4, el script llama a simple_switch con los argumentos de interfaz, puerto Thrift, --device-id y ruta al JSON compilado. Los hosts reciben sus direcciones IP, rutas por defecto y entradas ARP estáticas dentro del mismo script, de modo que el plano de control no necesita gestionar ARP durante los experimentos [2].

Dejar este terminal abierto durante toda la práctica:

```
sudo python3 mininet/topo.py
```

La salida observada en la práctica fue la siguiente:

```
*** Creating network  
*** Adding hosts:  
h1 h2 h11 h22  
*** Adding switches:  
s1 s2  
*** Adding links:  
(s1, h1) (s1, h11) (0.50Mbit) (0.50Mbit) (s1, s2) (s2, h2) (s2, h22)  
*** Configuring hosts  
h1 h2 h11 h22  
*** Starting controller  
  
*** Starting 2 switches  
s1 @ thrift-port 9090
```

```
cmd: simple_switch -i 1@s1-eth1 -i 2@s1-eth2 -i 3@s1-eth3
      --thrift-port 9090 --device-id 0 .../p4src/build/bmv2.json
s2 @ thrift-port 9091
cmd: simple_switch -i 3@s2-eth3 -i 1@s2-eth1 -i 2@s2-eth2
      --thrift-port 9091 --device-id 1 .../p4src/build/bmv2.json
```

La salida confirma que Mininet creó la red con los cuatro hosts y los dos switches, que el enlace cuello de botella s1–s2 tiene un ancho de banda de 0,50 Mbps, y que cada switch arrancó su proceso `simple_switch` escuchando en el puerto Thrift correspondiente. Los argumentos `--device-id 0` y `--device-id 1` distinguen las instancias BMv2 y evitan colisiones en los sockets IPC. El mensaje `Starting controller` sin ningún controlador real es un artefacto de la API de Mininet que no tiene efecto práctico, ya que el switch P4 no requiere controlador externo.

Paso 3 – Instalación de las reglas de reenvío

Una vez que los switches están en ejecución pero antes de que reciban ningún paquete, es necesario poblar sus tablas de reenvío mediante `simple_switch_CLI`. Cada archivo de comandos contiene tres entradas LPM: dos rutas /32 para los hosts conectados directamente al switch y una ruta /24 para la subred remota, que encamina el tráfico a través del puerto 3 (enlace inter-switch) [2].

En un segundo terminal, ejecutar:

```
simple_switch_CLI --thrift-port 9090 <s1-commands.txt
```

```
Obtaining JSON from switch...
Done
Control utility for runtime P4 table manipulation
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:          LPM-0a:00:01:01/32
action:             MyIngress.ipv4_forward
runtime data:       08:00:00:00:01:01  00:01
Entry has been added with handle 0
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:          LPM-0a:00:01:0b/32
action:             MyIngress.ipv4_forward
runtime data:       08:00:00:00:01:11  00:02
Entry has been added with handle 1
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:          LPM-0a:00:02:00/24
```

```
action:          MyIngress.ipv4_forward
runtime data:    08:00:00:00:02:00  00:03
Entry has been added with handle 2
```

```
simple_switch_CLI --thrift-port 9091 <s2-commands.txt
```

```
Obtaining JSON from switch...
Done
Control utility for runtime P4 table manipulation
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:      LPM-0a:00:02:02/32
action:         MyIngress.ipv4_forward
runtime data:    08:00:00:00:02:02  00:01
Entry has been added with handle 0
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:      LPM-0a:00:02:16/32
action:         MyIngress.ipv4_forward
runtime data:    08:00:00:00:02:22  00:02
Entry has been added with handle 1
RuntimeCmd: Adding entry to lpm match table MyIngress.ipv4_lpm
match key:      LPM-0a:00:01:00/24
action:         MyIngress.ipv4_forward
runtime data:    08:00:00:00:01:00  00:03
Entry has been added with handle 2
```

Cada línea Entry has been added with handle N confirma que simple_switch_CLI se comunicó correctamente con el proceso BMv2 a través del canal Thrift y que la entrada fue instalada en la tabla del switch emulado. La clave en formato LPM (0a:00:01:01/32) es la representación hexadecimal de la dirección IP 10.0.1.1 con máscara /32; 0a:00:01:0b es 10.0.1.11 y 0a:00:02:00/24 es la ruta agregada 10.0.2.0/24. La acción ipv4.forward toma como parámetros la MAC del siguiente salto (08:00:00:00:XX:XX) y el puerto de egreso en hexadecimal de dos bytes (00:01, 00:02, 00:03).

Paso 4 – Verificación de conectividad entre subredes

Con las tablas pobladas, se verifica la conectividad mediante el comando pingall de Mininet:

```
mininet>pingall
```

```
*** Ping: testing ping reachability
h1 -> h2 X h22
```



```
h2 -> h1 h11 X
h11 -> X h2 h22
h22 -> h1 X h11
*** Results: 33% dropped (8/12 received)
```

El resultado del 33 % de paquetes descartados es el comportamiento esperado y correcto. El programa P4 implementa únicamente reenvío L3 basado en la dirección IP destino. Los pares h1–h11 (ambos en s1, subred 10.0.1.0/24) y h2–h22 (ambos en s2, subred 10.0.2.0/24) no pueden comunicarse entre sí porque las solicitudes ARP son difusiones de capa 2 que no tienen una entrada en la tabla LPM. Los cuatro pares fallidos (h1→h11, h11→h1, h2→h22, h22→h2) representan exactamente el 33 % de los 12 pares posibles. Todos los pares entre subredes distintas (cruzando el enlace s1–s2) funcionan correctamente, confirmando que las rutas LPM y la reescritura de MACs operan como se espera. Este resultado ilustra una idea importante: una tasa de pérdida no nula en pingall no siempre indica un fallo; aquí confirma que el modelo de forwarding implementado es el correcto [2].

Paso 5 – Observación del marcado ECN

Esta es la etapa central del ejercicio. El objetivo es observar directamente cómo el switch P4 cambia el campo ECN de un paquete de 01 (ECT) a 11 (CE) cuando la cola del enlace cuello de botella supera el umbral definido en el programa.

Las herramientas estándar como ping o iperf no permiten enviar paquetes con un valor ECN preestablecido ni mostrar el campo TOS por paquete recibido. Los scripts send.py y receive.py basados en Scapy resuelven estas limitaciones: send.py construye paquetes UDP con el byte TOS puesto a 1 (ECN = 01, indicando que el emisor soporta ECN) y los transmite periódicamente; receive.py ejecuta un sniffer Scapy en la interfaz de destino y muestra el contenido completo del encabezado de cada paquete recibido, incluyendo el campo TOS, lo que hace visible la acción de marcado del switch [4]. Si Scapy no está disponible en la máquina virtual, instalarlo con `pip3 install scapy` antes de continuar.

Verificar que los scripts tienen permiso de ejecución antes de invocarlos desde los terminales de Mininet:

```
chmod +x send.py receive.py
```

Abrir terminales para h2 y h22 desde la CLI de Mininet:

```
mininet>xterm h2 h22
```

En el terminal de h22, iniciar el servidor UDP que absorberá el flujo de fondo:

```
h22# iperf -s -u -p 5001
```

En el terminal de h2, iniciar el receptor Scapy que imprime el byte TOS de cada paquete UDP entrante:

```
h2# python3 ./receive.py
```

Desde la CLI de Mininet, lanzar primero el flujo de fondo de alta tasa (h1 hacia h22) y luego los paquetes de prueba (h1 hacia h2):

```
mininet>h11 iperf -c 10.0.2.22 -p 5001 -t 15 -u -b 2M &
```

```
mininet>h1 ./send.py 10.0.2.2 "P4 is cool"20
```

La salida de h1 muestra el paquete enviado con tos = 0x1 (ECN-capable, ECT(0)):

```
###[ Ethernet ]###
  dst      = ff:ff:ff:ff:ff:ff
  src      = 08:00:00:00:01:01
  type     = IPv4
###[ IP ]###
  version  = 4
  ihl      = 5
  tos      = 0x1
  len      = 38
  ttl      = 64
  proto    = udp
  checksum = 0x63c3
  src      = 10.0.1.1
  dst      = 10.0.2.2
###[ UDP ]###
  sport    = 1234
  dport    = 4321
###[ Raw ]###
  load     = 'P4 is cool'
```

La salida observada en el terminal de h2 durante la congestión muestra tos = 0x3:

```
sniffing on eth0
got a packet
###[ Ethernet ]###
  dst      = 08:00:00:00:02:02
  src      = 08:00:00:00:02:00
  type     = IPv4
###[ IP ]###
  version  = 4
  ihl      = 5
  tos      = 0x3
  len      = 38
  ttl      = 62
  proto    = udp
  chksum   = 0x65c1
  src      = 10.0.1.1
  dst      = 10.0.2.2
###[ UDP ]###
  sport    = 1234
  dport    = 4321
###[ Raw ]###
  load     = 'P4 is cool'
```

El cambio de tos 0x1 a tos 0x3 es la evidencia directa del marcado ECN en el plano de datos. El paquete salió de h1 con el bit ECN en 01 (ECT, un solo bit activo) y llegó a h2 con ambos bits ECN en 11 (CE, Congestion Experienced). El switch s1, en cuyo puerto de egreso hacia s2 se concentra el tráfico de ambos flujos, realizó este marcado detectando que la profundidad de la cola superó el umbral de 10 celdas configurado en el programa P4 [4, 2].

Se observan además dos hechos relevantes en la captura de h2. Primero, el campo TTL disminuyó de 64 a 62, confirmando que el paquete atravesó dos switches (s1 y s2) y que la acción `ipv4_forward` decrementó el TTL en cada uno. Segundo, el campo `chksum` cambió de 0x63c3 a 0x65c1, lo que demuestra que el control `MyComputeChecksum` recalculó correctamente el checksum IPv4 tras la modificación del byte TOS.

Análisis del pipeline P4

El programa `ecn.p4` implementa reenvío IPv4 LPM con marcado ECN en el pipeline de egreso. A continuación se describe cada componente del pipeline y su justificación de diseño.

Estructura de encabezados – división del campo TOS

El campo TOS de 8 bits del encabezado IPv4 estándar se divide en dos subcampos independientes de bit₆ y bit₂:

```
header ipv4_t {  
    bit<4> version;  
    bit<4> ihl;  
    bit<6> diffserv; // servicios diferenciados (DSCP)  
    bit<2> ecn; // notificacion explicita de congestion  
    // ... otros campos ...  
}
```

Esta separación es un requisito del diseño: si el campo TOS se declarara como un único bit₈, el pipeline de egreso no podría modificar únicamente los bits ECN sin afectar los bits DSCP, y el cómputo de checksum tampoco podría referenciar cada subcampo por separado. La especificación P4-16 permite que los campos de un encabezado sean de cualquier ancho en bits, siempre que la suma total sea múltiplo de 8, lo que hace este diseño válido y eficiente [1].

Pipeline de ingreso – tabla LPM y reescritura de MACs

El control `MyIngress` aplica la tabla `ipv4_lpm`, que usa coincidencia LPM sobre el campo `hdr.ipv4.dstAddr`:

```
action ipv4_forward(macAddr_t dstAddr, egressSpec_t port) {  
    standard_metadata.egress_spec = port;  
    hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;  
    hdr.ethernet.dstAddr = dstAddr;  
    hdr.ipv4.ttl = hdr.ipv4.ttl - 1;  
}
```

La acción realiza tres operaciones propias del reenvío L3: asigna el puerto

de egreso, reescribe ambas MACs Ethernet (la MAC origen toma el valor de la MAC destino anterior, simulando el comportamiento de un router) y decrementa el TTL. Este modelo de reescritura de MACs es coherente con el comportamiento definido en la RFC 1812 [8].

Pipeline de egreso – detección de congestión y marcado ECN

El control MyEgress contiene la lógica central del ejercicio:

```
const bit<19>ECN_THRESHOLD = 10;

// En MyEgress.apply():
if (hdr.ipv4.ecn == 1 || hdr.ipv4.ecn == 2) {
    if (standard_metadata.enq_qdepth >= ECN_THRESHOLD) {
        mark_ecn(); // establece hdr.ipv4.ecn = 3
    }
}
```

El marcado ocurre en egreso y no en ingreso por una razón técnica fundamental: el metadato `standard_metadata.enq_qdepth` representa la profundidad de la cola en el momento en que el paquete fue encolado para el puerto de egreso, operación que ocurre entre los pipelines de ingreso y egreso. Por lo tanto, este valor no está disponible durante el procesamiento de ingreso. Este requisito es intrínseco a la arquitectura v1model e implementado por BMv2 [2].

La condición doble (`ecn == 1 OR ecn == 2`) implementa correctamente el estándar ECN: solo los paquetes ECN-capable (ECT(0) o ECT(1)) pueden ser marcados como CE. Los paquetes no ECN-capable (`ecn == 0`) no deben ser marcados; si la cola está llena y solo hay paquetes no ECN-capable, el switch debe descartarlos en lugar de marcarlos, comportamiento gestionado por el límite máximo de cola de BMv2 [4].

Recálculo del checksum IPv4

El control MyComputeChecksum recalcula el checksum cada vez que el paquete ha pasado por el pipeline completo. La llamada a `update_checksum` lista explícitamente los campos `diffserv` y `ecn` por separado:

```
update_checksum(
    hdr.ipv4.isValid(),
```

```
{ hdr.ipv4.version, hdr.ipv4.ihl,  
  hdr.ipv4.diffserv, hdr.ipv4.ecn,  
  hdr.ipv4.totalLen, hdr.ipv4.identification,  
  hdr.ipv4.flags, hdr.ipv4.fragOffset,  
  hdr.ipv4.ttl, hdr.ipv4.protocol,  
  hdr.ipv4.srcAddr, hdr.ipv4.dstAddr },  
  hdr.ipv4.hdrChecksum,  
  HashAlgorithm.csum16);
```

Este paso es obligatorio porque el checksum IPv4 cubre todo el encabezado IP. Cualquier modificación a un campo del encabezado (los bits ECN en egreso y el TTL en ingreso) invalida el checksum original, y los hosts receptores descartarían silenciosamente los paquetes cuyo checksum no coincida. El cambio verificado de 0x63c3 a 0x65c1 en la captura de h2 confirma que el recálculo opera correctamente. El algoritmo csum16 es el complemento a uno estándar definido en la RFC 791 [7].

Deparser

El deparser emite los encabezados en el orden declarado en la estructura headers: primero el encabezado Ethernet y luego el encabezado IPv4. El encabezado IPv4 emitido tiene los campos diffserv y ecn en las posiciones exactas del byte TOS original, de modo que el paquete resultante es un IPv4 estándar compatible con cualquier implementación de la pila de red [7, 1].

Ejercicio 2: Segmentación VLAN y marcado DSCP (Actividad del estudiante)

Contexto teórico

El estándar IEEE 802.1Q define el mecanismo de etiquetado VLAN para redes Ethernet. Una etiqueta 802.1Q es un encabezado de 4 bytes insertado entre las direcciones MAC y el tipo de protocolo: 2 bytes de TPID (0x8100) que identifican el frame como etiquetado, seguidos de 2 bytes de TCI que agrupan PCP (3 bits), DEI (1 bit) y el identificador de VLAN (VID, 12 bits) [9]. Los swit-

ches de acceso insertan la etiqueta al recibir tráfico sin etiquetar de un host y la eliminan antes de entregar el paquete al host destino. Los enlaces troncales (trunk) transportan tráfico de múltiples VLANs manteniendo las etiquetas.

La implementación de VLAN en P4 exige modelar explícitamente el encabezado 802.1Q en el programa, programar el parser para detectar el TPID 0x8100 y extraer el encabezado VLAN cuando está presente, y definir la lógica de inserción y eliminación en el pipeline de control. Esto ilustra uno de los principios fundamentales de P4: el programador tiene control total sobre qué encabezados reconoce el dispositivo y cómo los procesa, sin depender de que el hardware o el compilador los conozcan de antemano [1, 11].

El marcado DSCP complementa el etiquetado VLAN desde la perspectiva de calidad de servicio. El campo DSCP (6 bits) ocupa los bits más significativos del campo TOS del encabezado IPv4 y es definido por la arquitectura DiffServ de la RFC 2474 [5]. A diferencia de la etiqueta VLAN, que es un encabezado de capa 2 eliminado en el switch de egreso, el campo DSCP viaja en el encabezado IPv4 de extremo a extremo. Marcando el DSCP según la VLAN de origen en el switch de acceso, la red puede propagar la clasificación de tráfico más allá del dominio L2, habilitando políticas de QoS en routers intermedios que nunca vieron la etiqueta VLAN [5, 10].

Descripción de la topología y adaptaciones del ejercicio

El ejercicio está basado en el tutorial de P4 para ONOS incluido en el repositorio del curso, adaptado para operar con `simple_switch` y la CLI Thrift de BMv2 en lugar de la infraestructura gRPC/Stratum del tutorial original. Las adaptaciones incluyeron la corrección de incompatibilidades y errores del ejercicio original y la adición del marcado DSCP como reto de implementación del estudiante. En el `topo.py` adaptado, la llamada `net.staticArp()` se ubica explícitamente después de `net.start()`, garantizando que los namespaces de red estén creados cuando se programan las entradas ARP; llamarla antes de `net.start()` impediría programar las cachés ARP de los hosts. Al igual que en el ejercicio ECN, cada switch recibe un `—device-id` único para evitar colisiones de sockets IPC entre instancias BMv2.

La topología consiste en dos switches P4 y cuatro hosts divididos en dos VLANs. Los hosts h1 (VLAN 10) y h2 (VLAN 20) están conectados al switch s1 en los puertos 1 y 2 respectivamente. Los hosts h3 (VLAN 10) y h4 (VLAN 20) están conectados al switch s2 en los puertos 1 y 2 respectivamente. El puerto 3 de cada switch conecta con el otro switch y actúa como enlace troncal.

La asignación de direcciones es:

- h1: 10.10.10.1/29 (VLAN 10), MAC 00:00:00:00:00:01.
- h2: 20.20.20.1/26 (VLAN 20), MAC 00:00:00:00:00:02.
- h3: 10.10.10.2/29 (VLAN 10), MAC 00:00:00:00:00:03.
- h4: 20.20.20.2/26 (VLAN 20), MAC 00:00:00:00:00:04.

Todos los enlaces tienen parámetros de calidad de enlace: 5 Mbps de ancho de banda, 5 ms de retardo y 1 % de pérdida. Estos parámetros se configuran a través de TCLink en topo.py. El programa topo.py llama a net.staticArp() una vez que Mininet ya inició los hosts, pre-poblando las tablas ARP de todos los hosts y eliminando la resolución ARP como variable en el experimento; el programa P4 no implementa ningún mecanismo ARP, por lo que sin esta inicialización estática las solicitudes ARP en difusión serían descartadas por el switch.

Implementación P4 – estructura del programa

El programa main.p4 implementa el procesamiento VLAN bidireccional completo. A continuación se describe cada componente del pipeline.

Definición de encabezados y constantes

Los identificadores VLAN y los valores DSCP se declaran como constantes, y el encabezado VLAN 802.1Q se define explícitamente:

```
const bit<12>VLAN_10 = 0x00a; // VID 10 en hexadecimal
const bit<12>VLAN_20 = 0x014; // VID 20 en hexadecimal
const bit<6> DSCP_VLAN_10 = 10; // AF11 -- clase de trafico para V
const bit<6> DSCP_VLAN_20 = 20; // AF22 -- clase de trafico para V
```



```
header vlan_802_1q_t {
    bit<3> pri;    // priority code point (PCP)
    bit<1> cfi;    // drop eligible indicator (DEI)
    bit<12>vid;    // identificador de VLAN
    bit<16>ether_type; // protocolo encapsulado
}
```

Los identificadores VLAN se expresan en hexadecimal porque así los representa el runtime de BMv2 en la salida del CLI (00:0a para VLAN 10 y 00:14 para VLAN 20). Los valores DSCP se escogen de la tabla AF (Assured Forwarding): AF11 tiene el valor DSCP 10 y AF22 tiene el valor DSCP 20, según la RFC 2597 [6]. La distinción entre ambas clases permite que dispositivos QoS downstream traten el tráfico de cada VLAN con políticas diferenciadas.

Parser – detección del etiquetado

El parser examina el campo ether_type del encabezado Ethernet para determinar si el paquete lleva etiqueta VLAN:

```
state parse_ethernet {
    packet.extract(hdr.ethernet);
    transition select(hdr.ethernet.ether_type) {
        ETHERTYPE_IPV4: parse_ipv4; // trafico sin etiquetar
        ETHERTYPE_VLAN: parse_vlan; // trafico con etiqueta 802.1Q
        default: parse_ipv4;
    }
}

state parse_vlan {
    packet.extract(hdr.vlan_802_1q);
    transition select(hdr.vlan_802_1q.ether_type) {
        ETHERTYPE_IPV4: parse_ipv4;
        default: accept;
    }
}
```

El TPID (Tag Protocol Identifier) 0x8100 identifica un paquete Ethernet con etiqueta 802.1Q. Cuando el parser encuentra este valor, extrae el encabeza-

do VLAN y examina el campo ether_type interno del encabezado VLAN para continuar el parsing del protocolo encapsulado. Este mecanismo de selección anidada es la forma canónica de manejar encabezados opcionales en P4 [1].

Control de ingreso – tabla set_vlan_tag (acceso hacia trunk)

Cuando llega un paquete no etiquetado desde un host (ether_type = 0x0800), se aplica la tabla set_vlan_tag con clave de coincidencia sobre la MAC origen, y la acción add_vlan_tag realiza el etiquetado:

```
action add_vlan_tag(port_num_t port_num) {
    hdr.vlan_802_1q.setValid();
    // VID segun el puerto de ingreso
    if (standard_metadata.ingress_port == 1) {
        hdr.vlan_802_1q.vid = VLAN_10;
    } else if (standard_metadata.ingress_port == 2) {
        hdr.vlan_802_1q.vid = VLAN_20;
    }
    hdr.vlan_802_1q.pri = 0;
    hdr.vlan_802_1q.cfi = 0;
    hdr.vlan_802_1q.ether_type = hdr.ethernet.ether_type;
    hdr.ethernet.ether_type = ETHERTYPE_VLAN; // 0x8100
    // Marcado DSCP segun VLAN
    if (hdr.ipv4.isValid()) {
        if (standard_metadata.ingress_port == 1)
            hdr.ipv4.dscp = DSCP_VLAN_10;
        else if (standard_metadata.ingress_port == 2)
            hdr.ipv4.dscp = DSCP_VLAN_20;
    }
    standard_metadata.egress_spec = port_num;
}
```

La acción realiza varias operaciones en secuencia: activa el encabezado VLAN con setValid(), asigna el VID según el puerto físico de ingreso (política de VLAN basada en puerto), preserva el ether_type original del frame Ethernet desplazándolo al campo interno del encabezado VLAN, establece el ether_type del frame Ethernet en 0x8100 (el TPID), aplica el marcado DSCP al encabe-

zando IPv4 si está presente, y dirige el paquete al puerto troncal. Esta secuencia implementa fielmente el comportamiento de un switch de acceso 802.1Q [9, 1].

Control de ingreso – tabla `extract_vlan_tag` (trunk hacia acceso)

Cuando llega un paquete etiquetado desde el enlace troncal (`ether_type = 0x8100`), se aplica la tabla `extract_vlan_tag`:

```
table extract_vlan_tag {  
    key = {  
        hdr.ethernet.dst_addr: exact;  
        hdr.vlan_802_1q.vid: exact;  
    }  
    actions = { set_egress_port; @defaultonly drop; }  
}  
  
// En el bloque apply (tras extract_vlan_tag.apply()):  
hdr.ethernet.ether_type = hdr.vlan_802_1q.ether_type;  
hdr.vlan_802_1q.setInvalid();
```

La tabla usa coincidencia exacta sobre dos campos: la MAC destino y el VID. Solo existe una entrada por cada par (MAC, VID) válido dentro de la misma VLAN. Si llega un paquete cuyo par no tiene entrada (por ejemplo, un paquete de VLAN 10 dirigido a una MAC de VLAN 20), la acción por defecto marcada con `@defaultonly drop` lo descarta silenciosamente. Este mecanismo implementa el aislamiento VLAN completo en el plano de datos sin intervención del plano de control [1, 11].

Tras aplicar la tabla, el bloque `apply` restaura el campo `ether_type` del frame Ethernet desde el campo interno del encabezado VLAN (recuperando el valor `0x0800` para IPv4) y luego invalida el encabezado VLAN con `setInvalid()`. El `deparser` no emite los encabezados marcados como inválidos, de modo que el paquete entregado al host no contiene la etiqueta VLAN y es transparente para la pila de red del host.

Por qué el marcado DSCP no es redundante con el VID

La diferencia fundamental entre la etiqueta VLAN y el campo DSCP es el alcance. La etiqueta VLAN 802.1Q es eliminada antes de que el paquete llegue

al host destino: el pipeline llama a `hdr.vlan_802_1q.setInvalid()` en el switch de egreso, y el host receptor nunca ve el campo VID. La etiqueta VLAN es un mecanismo local entre switches, invisible para los extremos y para cualquier dispositivo fuera del dominio L2.

El campo DSCP, en cambio, forma parte del encabezado IPv4 y viaja de extremo a extremo. Cuando el paquete llega a h3, su encabezado IP conserva el valor `dscp=10`. Cualquier router, firewall o dispositivo QoS en la ruta posterior puede leer ese campo y aplicar políticas de tráfico sin necesidad de conocer la topología VLAN original. Este es el modelo DiffServ definido en la RFC 2474: clasificar el tráfico en el borde de la red (donde el switch de acceso tiene contexto VLAN) y marcarlo en el encabezado IP para que el núcleo de la red pueda actuar sin ese contexto [5, 6].

Checksum IPv4

El recálculo del checksum es necesario exclusivamente porque la acción `add_vlan_tag` modifica el campo `dscp` del encabezado IPv4. Sin la modificación DSCP, el ejercicio VLAN no alteraría ningún campo del encabezado IPv4 (no hay routing ni decremento de TTL), y el checksum original seguiría siendo válido. Con la modificación, el checksum se invalida porque DSCP forma parte del byte TOS que entra en el cálculo de suma de verificación. Los hosts receptores descartan silenciosamente los paquetes con checksum incorrecto [7, 5]:

```
update_checksum(  
    hdr.ipv4.isValid(),  
    { hdr.ipv4.version, hdr.ipv4.ihl,  
      hdr.ipv4.dscp, hdr.ipv4.ecn,  
      hdr.ipv4.total_len, hdr.ipv4.identification,  
      hdr.ipv4.flags, hdr.ipv4.frag_offset,  
      hdr.ipv4.ttl, hdr.ipv4.protocol,  
      hdr.ipv4.src_addr, hdr.ipv4.dst_addr },  
    hdr.ipv4.hdr_checksum,  
    HashAlgorithm.csum16);
```

Obsérvese que en `main.p4` los nombres de campo del encabezado IPv4 usan *snake_case* (`total_len`, `frag_offset`, `src_addr`, `dst_addr`, `hdr_checksum`), a diferencia de `ecn.p4` que usa *camelCase* (`totalLen`, `fragOffset`, `srcAddr`, `dstAddr`,

hdrChecksum). Esta diferencia no tiene impacto funcional; los campos ocupan las mismas posiciones en el encabezado IPv4 en ambos programas.

Paso 1 – Compilación del programa P4

Desde el directorio del ejercicio VLAN:

```
mkdir -p p4src/build  
p4c-bm2-ss --p4v 16 -o p4src/build/bmv2.json \  
--p4runtime-files p4src/build/p4info.txt p4src/main.p4
```

Una compilación sin errores confirma que el programa P4 es sintácticamente correcto y que todos los TODOs del estudiante han sido completados correctamente: que el encabezado `vlan_802_1q_t` tiene un ancho total de 32 bits (múltiplo de 8), que el estado `parse_vlan` hace una transición correcta hacia `parse_ipv4`, que la constante `DSCP_VLAN_10` tiene tipo `bit[6]` coherente con el campo `hdr.ipv4.dscp`, y que el bloque `ComputeChecksumImpl` cubre todos los campos del encabezado. Si alguno de estos TODOs estuviera mal resuelto, el compilador lo reportaría con un error de tipo o de alcance.

Paso 2 – Inicio de la topología Mininet

```
sudo python3 mininet/topo.py
```

```
*** Creating network  
*** Adding hosts:  
h1 h2 h3 h4  
*** Adding switches:  
s1 s2  
*** Adding links:  
(5.00Mbit 5ms delay 1.00000% loss) ... (s1, h1)  
(5.00Mbit 5ms delay 1.00000% loss) ... (s1, h2)  
(5.00Mbit 5ms delay 1.00000% loss) ... (s1, s2)  
(5.00Mbit 5ms delay 1.00000% loss) ... (s2, h3)  
(5.00Mbit 5ms delay 1.00000% loss) ... (s2, h4)  
*** Configuring hosts  
h1 h2 h3 h4  
*** Starting 2 switches  
s1 @ thrift-port 9090  
cmd: simple_switch -i 1@s1-eth1 -i 2@s1-eth2 -i 3@s1-eth3
```

```
--thrift-port 9090 --device-id 0 .../p4src/build/bmv2.json
s2 @ thrift-port 9091
cmd: simple_switch -i 3@s2-eth3 -i 1@s2-eth1 -i 2@s2-eth2
--thrift-port 9091 --device-id 1 .../p4src/build/bmv2.json
```

La salida confirma que todos los enlaces tienen las características de calidad configuradas (5 Mbps, 5 ms, 1 % de pérdida) y que ambos switches arrancaron con sus respectivos puertos Thrift. La llamada a `net.staticArp()` que precede al arranque de la CLI pre-pobla las tablas ARP de todos los hosts, de modo que el experimento VLAN mide únicamente el comportamiento del pipeline P4 y no la interacción con el mecanismo ARP.

Paso 3 – Instalación de las reglas de reenvío

En un segundo terminal, instalar las reglas en ambos switches:

```
simple_switch_CLI --thrift-port 9090 <s1-commands.txt
```

```
Obtaining JSON from switch...
Done
Control utility for runtime P4 table manipulation
RuntimeCmd: Adding entry to exact match table IngressPipeImpl.
    set_vlan_tag
match key:          EXACT-00:00:00:00:00:01
action:             IngressPipeImpl.add_vlan_tag
runtime data:       00:03
Entry has been added with handle 0
RuntimeCmd: Adding entry to exact match table IngressPipeImpl.
    set_vlan_tag
match key:          EXACT-00:00:00:00:00:02
action:             IngressPipeImpl.add_vlan_tag
runtime data:       00:03
Entry has been added with handle 1
RuntimeCmd: Adding entry to exact match table IngressPipeImpl.
    extract_vlan_tag
match key:          EXACT-00:00:00:00:00:01      EXACT-00:0a
action:             IngressPipeImpl.set_egress_port
runtime data:       00:01
Entry has been added with handle 0
RuntimeCmd: Adding entry to exact match table IngressPipeImpl.
    extract_vlan_tag
match key:          EXACT-00:00:00:00:00:02      EXACT-00:14
action:             IngressPipeImpl.set_egress_port
runtime data:       00:02
Entry has been added with handle 1
```

```
simple_switch_CLI --thrift-port 9091 <s2-commands.txt
```

```
Obtaining JSON from switch...
Done
Control utility for runtime P4 table manipulation
RuntimeCmd: Adding entry to exact match table IngressPipeImpl.
    set_vlan_tag
match key:          EXACT-00:00:00:00:00:03
action:             IngressPipeImpl.add_vlan_tag
runtime data:       00:03
Entry has been added with handle 0
RuntimeCmd: Adding entry to exact match table IngressPipeImpl.
    set_vlan_tag
match key:          EXACT-00:00:00:00:00:04
action:             IngressPipeImpl.add_vlan_tag
runtime data:       00:03
Entry has been added with handle 1
RuntimeCmd: Adding entry to exact match table IngressPipeImpl.
    extract_vlan_tag
match key:          EXACT-00:00:00:00:00:03      EXACT-00:0a
action:             IngressPipeImpl.set_egress_port
runtime data:       00:01
Entry has been added with handle 0
RuntimeCmd: Adding entry to exact match table IngressPipeImpl.
    extract_vlan_tag
match key:          EXACT-00:00:00:00:00:04      EXACT-00:14
action:             IngressPipeImpl.set_egress_port
runtime data:       00:02
Entry has been added with handle 1
```

Las reglas de `set_vlan_tag` usan coincidencia exacta sobre la MAC origen del paquete y dirigen el tráfico al puerto 3 (parámetro 00:03). Las reglas de `extract_vlan_tag` usan dos claves de coincidencia: la MAC destino y el VID en formato hexadecimal (00:0a = 10 para VLAN 10, 00:14 = 20 para VLAN 20). El CLI instala dos tablas con responsabilidades diferentes y complementarias: `set_vlan_tag` decide la salida al troncal y construye el encabezado 802.1Q, mientras `extract_vlan_tag` realiza el forwarding final y su acción por defecto de descarte implementa el aislamiento entre VLANs.

Paso 4 – Verificación del aislamiento VLAN y conectividad intra-VLAN

Con las tablas pobladas, se ejecuta un test de conectividad completo para verificar simultáneamente que la conectividad intra-VLAN funciona y que el aislamiento cross-VLAN opera correctamente:

```
mininet>pingall
```

```
*** Ping: testing ping reachability
h1 -> X h3 X
h2 -> X X h4
h3 -> h1 X X
h4 -> X h2 X
*** Results: 66% dropped (4/12 received)
```

El resultado del 66 % de paquetes descartados es el comportamiento esperado y correcto. De los 12 pares posibles, exactamente 4 tienen conectividad: $h1 \leftrightarrow h3$ (ambos en VLAN 10) y $h2 \leftrightarrow h4$ (ambos en VLAN 20). Los 8 pares restantes se pueden clasificar en dos categorías. La primera son pares en VLANs distintas ($h1-h4$ y $h2-h3$): la acción @defaultonly drop de la tabla extract_vlan_tag descarta el paquete en el plano de datos porque no existe ninguna entrada que case con el par (MAC_destino, VID). La segunda son pares en la misma VLAN pero en el mismo switch ($h1-h2$ en s1, $h3-h4$ en s2): el pipeline fue diseñado como acceso-troncal-acceso y no implementa un camino local host-a-host dentro del mismo switch sin pasar por el troncal [9].

Este resultado demuestra que el aislamiento VLAN es completo y opera exclusivamente en el plano de datos P4, sin ningún mecanismo de filtrado en el plano de control. Leído correctamente, pingall funciona aquí como una verificación global del patrón de conectividad impuesto por el pipeline, permitiendo comprobar si la segmentación observada coincide exactamente con el diseño esperado.

Paso 5 – Verificación del marcado DSCP

Para verificar que el campo DSCP fue marcado correctamente por el pipeline P4, se usa tcpdump con la opción -v en los hosts receptores mientras se

genera tráfico:

```
mininet>xterm h3 h4
```

En el terminal de h3 (VLAN 10, DSCP esperado = 10):

```
h3# tcpdump -i eth0 -v -c 10
```

En el terminal de h4 (VLAN 20, DSCP esperado = 20):

```
h4# tcpdump -i eth0 -v -c 10
```

Desde la CLI de Mininet, generar tráfico de prueba:

```
mininet>h1 ping h3 -c 5
```

```
mininet>h2 ping h4 -c 5
```

En la captura de h3 se espera observar el campo TOS con valor 0x28 (decimal 40), ya que DSCP = 10 ocupa los 6 bits superiores del byte TOS y los dos bits ECN son cero ($10 \times 4 = 40$). En la captura de h4 se espera TOS = 0x50 (decimal 80), correspondiente a DSCP = 20 ($20 \times 4 = 80$). La etiqueta VLAN no aparece en ninguna de las capturas porque el switch la eliminó antes de entregar el paquete al host: `hdr.vlan_802_1q.setInvalid()` en el bloque `apply` evita que el `deparser` la emita.

El correcto funcionamiento de este paso se sustenta en dos evidencias ya verificadas: el CLI instaló las reglas de `set_vlan_tag` en ambos switches con las constantes VLAN correctas (confirmado por los valores 00:0a y 00:14 en el paso 3), y el programa P4 compilado contiene la asignación efectiva de `DSCP_VLAN_10` y `DSCP_VLAN_20` junto con el recálculo de checksum. La conectividad intra-VLAN ya verificada en el paso anterior confirma además que el `deparser` emite correctamente el paquete sin la etiqueta VLAN [9, 5].

Discusión comparativa

Nivel de pipeline donde ocurre la modificación

Los dos ejercicios ilustran que no todas las modificaciones de encabezados pueden ni deben realizarse en el mismo punto del pipeline P4. En el ejercicio ECN, el marcado solo puede ocurrir en el pipeline de egreso porque el metadato `enq.qdepth` no está disponible en ingreso: el paquete todavía no ha sido encolado para ningún puerto de salida. En el ejercicio VLAN, la inserción y extracción del encabezado 802.1Q ocurren en ingreso porque dependen únicamente del puerto de ingreso y de los campos del paquete, sin necesidad de información de estado del buffer de salida.

Este contraste ilustra la diferencia entre decisiones de reenvío basadas en contexto de llegada (ingreso) y decisiones basadas en condiciones de la red en el momento de la transmisión (egreso). La arquitectura v1model de P4 separa claramente ambos pipelines, lo que obliga al programador a razonar explícitamente sobre en qué punto del procesamiento está disponible cada pieza de información [1, 2].

Alcance de las modificaciones: enlace local versus extremo a extremo

Ambos ejercicios modifican el encabezado IPv4, pero con alcances y propósitos distintos. El ejercicio ECN modifica el campo `ecn` en el encabezado IP, y ese cambio viaja hasta el receptor: `h2` observa en la captura Scapy el nuevo valor de TOS. El receptor puede entonces notificar al emisor para reducir la tasa de envío, completando el circuito de retroalimentación de control de congestión definido en la RFC 3168 [4].

En el ejercicio VLAN, el switch modifica tanto el encabezado Ethernet (insertando o eliminando la etiqueta VLAN) como el encabezado IPv4 (marcando el campo DSCP). La etiqueta VLAN tiene alcance local: es eliminada en el switch de destino y el host nunca la ve. El marcado DSCP tiene alcance extremo a extremo: el host receptor ve el valor DSCP en el encabezado IPv4. Esta distinción entre mecanismos de capa 2 y campos de capa 3 es fundamental

en el diseño de redes con diferenciación de servicios [5, 9].

Modelo de reenvío: LPM versus tabla exacta

El ejercicio ECN usa coincidencia LPM sobre la dirección IP destino, lo que permite expresar tanto rutas host (/32) como rutas de subred (/24) en la misma tabla con una sola operación de lookup. Este modelo es el estándar en dispositivos de capa 3 (routers) y requiere reescritura de MACs en cada salto para mantener la coherencia del encabezado Ethernet [8].

El ejercicio VLAN usa coincidencia exacta sobre la MAC destino (y el VID en la tabla de extracción), lo que es el modelo natural para switches de capa 2. No se modifica ningún campo del encabezado IP relacionado con el routing (ni la IP destino ni el TTL), lo que mantiene el paquete idéntico desde la perspectiva de la capa 3. Esta diferencia de modelo de reenvío entre los dos ejercicios permite al estudiante comparar directamente cómo P4 implementa comportamientos de capa 2 y de capa 3 dentro del mismo framework de programación [1, 11].

Recálculo de checksum en ambos ejercicios

Ambos ejercicios requieren recalcular el checksum IPv4. En el ejercicio ECN, la causa es doble: se modifican los bits ECN (parte del byte TOS) y se decrementa el TTL. En el ejercicio VLAN, solo se modifica el campo DSCP (parte del byte TOS); el TTL permanece intacto porque no hay routing. En ambos casos, el bloque `update_checksum` lista los campos `dscp` (o `diffserv`) y `ecn` de forma separada porque el encabezado IPv4 ya no tiene un campo `tos` unificado de 8 bits, sino dos campos independientes. Esta división de campo es un patrón de diseño estándar en programas P4 que necesitan acceso independiente a DSCP y ECN [1, 7].

Respuestas a las preguntas de discusión

Pregunta 1. En el pipeline ECN se modifican el campo `ecn` (en egreso) y el campo `ttl` (en ingreso, durante la acción `ipv4_forward`). En el pipeline VLAN se modifica el campo `dscp` (en ingreso, durante `add_vlan_tag`). En ambos casos el checksum IPv4 debe recalcularse porque el algoritmo de checksum definido en la RFC 791 cubre todos los campos del encabezado IP como una suma en complemento a uno. Cualquier modificación de un bit en el encabezado altera la suma y hace el checksum previo inválido. Si no se recalcula, el host receptor descarta el paquete sin notificación al emisor, lo que resulta en pérdidas silenciosas de tráfico [7].

Pregunta 2. El marcado ECN no puede realizarse en ingreso porque el valor de `standard_metadata.enq_qdepth` no está definido en ese punto del pipeline: el paquete todavía no ha sido encolado para ningún puerto de salida. El encolado ocurre entre el pipeline de ingreso y el de egreso, y solo entonces se registra la profundidad de la cola en los metadatos del paquete. El marcado DSCP en el ejercicio VLAN, en cambio, depende únicamente del puerto de ingreso (disponible desde el primer ciclo de procesamiento), por lo que puede realizarse en cualquier punto del pipeline; se elige ingreso por coherencia con el resto de la lógica de reenvío [1, 2].

Pregunta 3. Un campo local a un segmento de enlace, como la etiqueta VLAN 802.1Q, existe solo en el encabezado de capa 2 y tiene validez únicamente dentro del dominio L2. El switch de egreso elimina la etiqueta antes de entregar el paquete, de modo que el host receptor y cualquier dispositivo fuera del dominio L2 nunca ven ese campo. Un campo extremo a extremo, como DSCP o ECN, forma parte del encabezado de capa 3 (IP) y viaja sin modificación a través de todos los dominios de red hasta el destino final (salvo que algún dispositivo intermedio lo modifique explícitamente). Esta diferencia explica el comportamiento de los puertos de acceso (inserción y eliminación de etiqueta) y de los puertos troncales (preservación de etiqueta): el trunk es el mecanismo para transportar la clasificación VLAN entre switches dentro del mismo dominio L2, mientras que el DSCP es el mecanismo para llevar la clasificación más

allá de ese dominio [9, 5].

Conclusión

Este laboratorio demostró experimentalmente dos mecanismos avanzados de procesamiento de paquetes implementados en el plano de datos programable con P4. El ejercicio ECN mostró cómo un switch puede participar activamente en el control de congestión de la red señalizando a los extremos sin descartar tráfico, mediante la lectura del metadato de profundidad de cola en el pipeline de egreso y la modificación del campo ECN del encabezado IPv4. La observación directa del cambio de TOS 0x1 a TOS 0x3 en los paquetes capturados por receive.py valida que el pipeline opera correctamente bajo condiciones de congestión inducida.

El ejercicio VLAN demostró cómo un switch programable puede implementar segmentación de red de capa 2 completa mediante la inserción y eliminación de encabezados 802.1Q, con aislamiento de tráfico inter-VLAN garantizado por el plano de datos a través de la acción @defaultonly drop. La adición del marcado DSCP ilustró la integración de mecanismos L2 y L3 en un único pipeline P4, conectando la clasificación de red local (VLAN) con los mecanismos de diferenciación de servicios de alcance global (DiffServ). El patrón de conectividad del pingall (66 % de pérdida, con exactamente los 4 pares intra-VLAN funcionando) valida que el aislamiento opera exclusivamente en el plano de datos sin intervención del plano de control.

Juntos, ambos ejercicios refuerzan los principios centrales de P4 como lenguaje de descripción de planos de datos: la independencia de protocolo (cualquier encabezado puede ser definido y manipulado), la separabilidad de campos (los campos de un encabezado pueden subdividirse según las necesidades del programa) y la separación de pipelines (ingreso y egreso tienen acceso a diferentes metadatos de la arquitectura v1 model). La coherencia de la infraestructura de desarrollo (mismo flujo p4c/topo.py/simple_switch_CLI en ambos ejercicios) permitió al estudiante concentrar su atención en los conceptos de red en lugar de en cambios de herramienta. Estos conceptos son

la base para ejercicios posteriores de mayor complejidad, como telemetría in-band (INT), firewalls en el plano de datos y clasificación de tráfico con machine learning embebido [1, 10, 12].

Referencias

- [1] P4.org, “P4 Language Specification (P4-16),” ver. 1.2.x, 2021–2024. [En línea]. Disponible en: <https://p4.org/p4-spec/>
- [2] P4.org, “Behavioral Model (BMv2) Documentation,” 2020–2024. [En línea]. Disponible en: <https://github.com/p4lang/behavioral-model>
- [3] P4.org, “P4 Tutorials,” repositorio oficial de ejercicios de referencia, 2024. [En línea]. Disponible en: <https://github.com/p4lang/tutorials>
- [4] K. Ramakrishnan, S. Floyd y D. Black, “The Addition of Explicit Congestion Notification (ECN) to IP,” RFC 3168, IETF, Sep. 2001.
- [5] K. Nichols, S. Blake, F. Baker y D. Black, “Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers,” RFC 2474, IETF, Dic. 1998.
- [6] J. Heinanen, F. Baker, W. Weiss y J. Wroclawski, “Assured Forwarding PHB Group,” RFC 2597, IETF, Jun. 1999.
- [7] J. Postel, “Internet Protocol,” RFC 791, IETF, Sep. 1981.
- [8] F. Baker, Ed., “Requirements for IP Version 4 Routers,” RFC 1812, IETF, Jun. 1995.
- [9] IEEE, “IEEE Std 802.1Q-2022 – IEEE Standard for Local and Metropolitan Area Networks – Bridges and Bridged Networks,” IEEE, Oct. 2022.
- [10] J. F. Botero, “Data Plane Programmability – Introduction and Applications,” Univ. de Antioquia / Univ. of Antwerp, notas de curso Internet del Futuro, 2020.

- //////
- [11] J. F. Botero, "Data Plane Programmability – P4 Language," Univ. de Antioquia / Univ. of Antwerp, notas de curso Internet del Futuro, 2020.
- [12] C. Garzón et al., "P4 Cybersecurity Solutions: Taxonomy and Open Challenges," *IEEE Access*, vol. 12, 2024.